

FastAPI

Complete Developer Handbook

Stages 1–5 | Concepts · Code · Edge Cases

Built during hands-on learning sessions

Stage 1 — Setup & Core Concepts

1.1 What is FastAPI?

FastAPI is a Python web framework for building APIs. It is fast to write, fast to run, validates data automatically, and generates interactive documentation (Swagger UI) for free.

Uvicorn is the ASGI server that runs your FastAPI app. FastAPI is the engine; Uvicorn is the car — you need both.

1.2 Installation

```
pip install fastapi uvicorn
```

Tip: Always use a virtual environment per project. Run: `python3 -m venv venv && source venv/bin/activate`

1.3 Running the App

```
uvicorn main:app --reload
```

main = filename, **app** = FastAPI instance, **--reload** = auto-restart on save (dev only).

Warning: Never use `--reload` in production. It is slow and insecure.

1.4 Your First Route

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello, World!"}
```

FastAPI automatically converts the returned dict to JSON.

1.5 Path Parameters

Path parameters are variables embedded in the URL itself.

```
@app.get("/greet/{name}")
def greet_user(name: str):
    return {"message": f"Hello, {name}!"}
```

FastAPI auto-converts types. If you declare **user_id: int**, passing "42" in the URL becomes integer 42 automatically. Passing "abc" returns a 422 error automatically.

1.6 Query Parameters

Any function parameter not in the path is treated as a query parameter (after ? in the URL).

```
@app.get("/products")
def get_products(category: str, in_stock: bool = True):
    return {"category": category, "in_stock": in_stock}

# URL: /products?category=electronics&in_stock=false
```

Note: Default values make parameters optional. Without a default, FastAPI requires them.

1.7 HTTP Status Codes

Code	Meaning	When to Use
200	OK	Default success
201	Created	New resource created (POST)
400	Bad Request	Client sent invalid data
401	Unauthorized	Not authenticated
403	Forbidden	Authenticated but no permission
404	Not Found	Resource does not exist
422	Unprocessable Entity	Validation failed (auto by FastAPI)
500	Internal Server Error	Something broke on your end

1.8 Raising HTTP Exceptions

```
from fastapi import HTTPException

@app.get("/users/{user_id}")
def get_user(user_id: int):
    if user_id != 1:
        raise HTTPException(status_code=404, detail="User not found")
    return {"id": 1, "name": "John"}
```

1.9 Auto Docs (Swagger UI)

Visit <http://127.0.0.1:8000/docs> for interactive Swagger UI. FastAPI generates this automatically from your code — zero extra work.

Tip: FastAPI infers possible responses from your type hints and decorators without you running anything.

1.10 Virtual Environment

```
python3 -m venv venv
source venv/bin/activate      # Mac/Linux
venv\Scripts\activate        # Windows
```

```
pip install fastapi uvicorn  
pip freeze > requirements.txt
```

Note: Always use a venv. `pip freeze > requirements.txt` saves dependencies so others can recreate your setup.

Stage 2 — Pydantic Models

2.1 What is Pydantic?

Pydantic is a data validation library. You define the shape of your data as a Python class. FastAPI uses it to validate incoming requests automatically.

Think of it like TypeScript interfaces or MongoDB models — but enforces validation at runtime.

2.2 Basic Model

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int
    email: str

@app.post("/users", status_code=201)
def create_user(user: User):
    return {"message": f"User {user.name} created!", "data": user}
```

Invalid data (wrong types, missing fields) automatically returns a **422 Unprocessable Entity**.

2.3 Optional Fields & Default Values

```
from typing import Optional

class Product(BaseModel):
    name: str
    price: float
    in_stock: bool = True # optional, defaults to True
    description: Optional[str] = None # optional, defaults to None
    discount: Optional[float] = 0.0
```

Note: Optional fields MUST have a default value. Without it, FastAPI still requires them.

Optional[str] = None means value can be string or None. Returns null in JSON when absent.

2.4 Nested Models

```
class Address(BaseModel):
    street: str
    city: str
    pincode: str

class Order(BaseModel):
    order_id: int
    customer_name: str
```

```

    address: Address    # nested model

@app.post("/orders", status_code=201)
def create_order(order: Order):
    return {"message": "Order placed!", "data": order}

```

FastAPI validates both levels automatically. Same concept as nested interfaces in TypeScript.

2.5 Response Models

Control exactly what gets sent back in the response. Use to hide sensitive fields like passwords.

```

class UserIn(BaseModel):
    name: str
    email: str
    password: str    # received from client

class UserOut(BaseModel):
    id: int
    name: str
    email: str
    is_active: bool

    class Config:
        from_attributes = True # required to read SQLAlchemy objects

@app.post("/users", response_model=UserOut, status_code=201)
def create_user(user: UserIn):
    return user # password is automatically stripped!

```

2.6 Edge Cases — Pydantic

Field name matching: UserOut field names must match the returned object — mapped by name.

Required fields in response model: All required fields in UserOut must exist in what you return.

Type coercion: Compatible types convert automatically (float to int). Incompatible types throw errors.

from_attributes = True: Always add to schemas used as response_model with SQLAlchemy objects.

Warning: In Pydantic v2, 'class Config' is deprecated. Use: model_config = ConfigDict(from_attributes=True)

Stage 3 — CRUD API with a Database

3.1 Project Structure

```
fastapi-learning/
■■■ main.py          # routes only
■■■ database.py      # DB connection, session, Base
■■■ models.py        # SQLAlchemy table definitions
■■■ schemas.py       # Pydantic request/response models
■■■ requirements.txt
```

Tip: Separation of concerns — each file has one job.

3.2 database.py

```
from sqlalchemy.orm import DeclarativeBase, sessionmaker
from sqlalchemy import create_engine

engine = create_engine('sqlite:///app.db')

SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db          # provides session to route
    finally:
        db.close()       # always closes, even on error
```

Note: autocommit=False: manually control commits. autoflush=False: changes sent only on commit.

3.3 models.py

```
from sqlalchemy.orm import Mapped, mapped_column
from database import Base

class User(Base):
    __tablename__ = 'users'

    id: Mapped[int] = mapped_column(primary_key=True) # auto-increments
    name: Mapped[str] = mapped_column()
    email: Mapped[str] = mapped_column(unique=True)
    is_active: Mapped[bool] = mapped_column(default=True)
    password: Mapped[str] = mapped_column()
```

Classes inheriting Base are registered automatically. **create_all** reads this registry.

3.4 schemas.py

```
from pydantic import BaseModel

class UserCreate(BaseModel): # what client sends
    name: str
    email: str
    password: str

class UserOut(BaseModel):    # what API returns
    id: int
    name: str
    email: str
    is_active: bool

    class Config:
        from_attributes = True
```

SQLAlchemy model = database layer. **Pydantic schema** = API layer. Always separate them.

3.5 Dependency Injection

Dependency injection runs setup/teardown code automatically before/after a route. Perfect for DB sessions.

```
from fastapi import Depends

@app.get("/users")
def get_users(db: Session = Depends(get_db)):
    # db session is ready here, closed automatically after
    return db.query(User).all()
```

3.6 CRUD Routes

```
# CREATE
@app.post('/users', response_model=UserOut, status_code=201)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    new_user = User(name=user.name, email=user.email, password=user.password)
    db.add(new_user)
    db.commit()
    db.refresh(new_user) # syncs DB-generated fields (id) back to object
    return new_user

# READ ALL
@app.get('/users', response_model=list[UserOut])
def get_users(db: Session = Depends(get_db)):
    return db.query(User).all()

# READ ONE
@app.get('/users/{user_id}', response_model=UserOut)
def get_user(user_id: int, db: Session = Depends(get_db)):
```

```

    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    return user

# UPDATE
@app.put('/users/{user_id}', response_model=UserOut)
def update_user(user_id: int, data: UserCreate, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    user.name = data.name
    user.email = data.email
    db.commit()
    db.refresh(user)
    return user

# DELETE
@app.delete('/users/{user_id}')
def delete_user(user_id: int, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.id == user_id).first()
    if not user:
        raise HTTPException(status_code=404, detail="User not found")
    db.delete(user)
    db.commit()
    return {"message": "User deleted successfully"}

```

3.7 CRUD HTTP Methods

Operation	Method	Route	Status Code
Create	POST	/users	201
Read All	GET	/users	200
Read One	GET	/users/{id}	200
Update	PUT	/users/{id}	200
Delete	DELETE	/users/{id}	200

3.8 Edge Cases — Database

create_all: Only creates tables that don't exist. Never modifies existing ones. Use Alembic for schema changes.

db.refresh(): Syncs DB-generated values (id, defaults) back to your Python object after commit.

Empty list response: Return [] for empty collections. Use HTTPException for error messages.

Never mix return types: If response_model is list[UserOut], always return a list — never a dict.

Stage 4 — Auth, Middleware & Async

4.1 Async/Await

FastAPI supports both sync and async routes. Only use `async def` when you have `await` inside the function.

```
# sync – use for SQLAlchemy routes
def get_users(db: Session = Depends(get_db)):
    return db.query(User).all()

# async – use when calling external APIs
async def call_external():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.example.com/data")
    return response.json()
```

Note: SQLAlchemy sync operations inside async routes work fine in FastAPI. Don't force async where it doesn't belong.

4.2 Installing Auth Packages

```
pip install python-jose[cryptography] passlib bcrypt==4.0.1
```

Warning: Use `bcrypt==4.0.1` specifically. Newer versions have compatibility issues with `passlib`.

4.3 auth.py — Password Hashing & JWT

```
from passlib.context import CryptContext
from datetime import datetime, timedelta, timezone
from jose import JWTError, jwt
from dotenv import load_dotenv
import os

load_dotenv()
SECRET_KEY: str = os.getenv("SECRET_KEY", '')
ALGORITHM: str = os.getenv("ALGORITHM", 'HS256')
ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES") or 30)

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    # ORDER MATTERS: plain first, hashed second
    return pwd_context.verify(plain_password, hashed_password)

def create_access_token(data: dict):
    to_encode = data.copy() # never mutate original dict
```

```

# datetime.now(timezone.utc) – utcnow() is deprecated in Python 3.12+
expire = datetime.now(timezone.utc) + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
to_encode.update({"exp": expire}) # "exp" is a standard JWT claim
return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def decode_access_token(token: str):
    try:
        return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    except JWTError:
        return None # catches both invalid and expired tokens

```

4.4 oauth2.py — Protecting Routes

```

from fastapi import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer
from auth import decode_access_token

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")

def get_current_user(token: str = Depends(oauth2_scheme)):
    payload = decode_access_token(token)
    if payload is None:
        raise HTTPException(status_code=401, detail="Could not validate credentials")
    return payload

```

Add `current_user = Depends(get_current_user)` to any route you want to protect.

4.5 Auth Routes

```

@app.post('/register', response_model=UserOut, status_code=201)
def register_user(user: UserCreate, db: Session = Depends(get_db)):
    existing = db.query(User).filter(User.email == user.email).first()
    if existing:
        raise HTTPException(status_code=400, detail="Email already registered")
    hashed = hash_password(user.password)
    new_user = User(name=user.name, email=user.email, password=hashed)
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return new_user

@app.post('/login', response_model=UserToken, status_code=200)
def login(user: UserLogin, db: Session = Depends(get_db)):
    db_user = db.query(User).filter(User.email == user.email).first()
    if not db_user:
        raise HTTPException(status_code=400, detail="User does not exist")
    if not verify_password(user.password, db_user.password):
        raise HTTPException(status_code=401, detail="Incorrect password")
    token = create_access_token({"sub": db_user.email})

```

```
return {"access_token": token}
```

4.6 Auth Flow

```
Register -> hash password -> save to DB
Login    -> verify password against DB hash -> issue JWT token
Protected -> extract token from header -> verify -> grant access
```

4.7 Middleware

Code that runs on every request before/after your route. Common uses: logging, CORS, response time.

```
import time
from fastapi import Request

@app.middleware("http")
async def log_requests(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request) # runs the actual route
    process_time = time.time() - start_time
    print(f"{request.method} {request.url} - {process_time:.4f}s")
    return response
```

4.8 CORS Middleware

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"], # whitelist frontend URL
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Warning: Never use `allow_origins=["*"]` in production. Always specify exact domains.

4.9 Edge Cases — Auth

verify_password order matters: plain_password first, hashed_password (from DB) second.

JWT payload is not encrypted: It is base64 encoded. Never put passwords or sensitive data in JWT.

401 vs 403: 401 = not authenticated (no/invalid token). 403 = authenticated but no permission.

data.copy(): Always copy dict before modifying in `create_access_token`.

SECRET_KEY in .env: Never hardcode or commit to git. Store in environment variables.

Stage 5 — Testing, Structure & Deployment

5.1 Alembic — Database Migrations

Alembic safely updates existing database tables without losing data. Think of it as git for your database schema.

```
pip install alembic
alembic init alembic
```

Configure alembic.ini:

```
sqlalchemy.url = sqlite:///./app.db
```

Configure alembic/env.py:

```
from models import Base
target_metadata = Base.metadata
```

Common Alembic commands:

```
alembic revision --autogenerate -m "message" # generate migration
alembic upgrade head                        # apply all pending
alembic downgrade -1                       # rollback one migration
alembic upgrade head                        # must apply before creating new
```

Note: create_all only creates new tables. It never modifies existing ones. Always use Alembic for schema changes in production.

5.2 Environment Variables

```
# .env file (never commit to git)
SECRET_KEY=mysupersecretkey
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=30
DATABASE_URL=sqlite:///./app.db
```

```
# auth.py
from dotenv import load_dotenv
import os

load_dotenv()
SECRET_KEY: str = os.getenv("SECRET_KEY", '')
ALGORITHM: str = os.getenv("ALGORITHM", 'HS256')
ACCESS_TOKEN_EXPIRE_MINUTES = int(os.getenv("ACCESS_TOKEN_EXPIRE_MINUTES") or 30)

# For critical secrets – crash loudly if missing:
assert SECRET_KEY, "SECRET_KEY is not set in .env"
```

Warning: Add .env to .gitignore. Never commit secrets to version control.

5.3 Project Structure with APIRouter

```

fastapi-learning/
├── routers/
│   ├── users.py      # all user routes
│   └── auth.py       # register/login routes
├── tests/
│   ├── __init__.py
│   └── test_users.py
├── alembic/
├── main.py           # app setup only, no routes
├── database.py
├── models.py
├── schemas.py
├── auth.py
├── oauth2.py
├── .env
└── requirements.txt

```

```

# routers/users.py
from fastapi import APIRouter

router = APIRouter(prefix="/users", tags=["Users"])

@router.get("/", response_model=list[UserOut])
def get_users(db: Session = Depends(get_db)):
    return db.query(User).all()

# main.py
from routers import users, auth
app.include_router(users.router)
app.include_router(auth.router)

```

Tip: prefix removes repetition. tags groups routes in Swagger UI.

5.4 Testing with Pytest

```

pip install pytest httpx

```

```

# tests/test_users.py
from fastapi.testclient import TestClient
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from main import app
from database import Base, get_db
import pytest

TEST_DATABASE_URL = "sqlite:///./test.db"
test_engine = create_engine(TEST_DATABASE_URL)
TestSessionLocal = sessionmaker(bind=test_engine, autocommit=False, autoflush=False)

def override_get_db():
    db = TestSessionLocal()
    try:

```

```

        yield db
    finally:
        db.close()

app.dependency_overrides[get_db] = override_get_db

@pytest.fixture(autouse=True, scope="session")
def reset_db():
    Base.metadata.drop_all(bind=test_engine)
    Base.metadata.create_all(bind=test_engine)
    yield

client = TestClient(app)

def test_register_user():
    response = client.post("/auth/register", json={
        "name": "Test User",
        "email": "test@example.com",
        "password": "123456"
    })
    assert response.status_code == 201
    assert response.json()["email"] == "test@example.com"

def test_login_user():
    response = client.post("/auth/login", json={
        "email": "test@example.com",
        "password": "123456"
    })
    assert response.status_code == 200
    assert "access_token" in response.json()

def test_get_users_without_token():
    response = client.get("/users/")
    assert response.status_code == 401

```

```
pytest tests/ -v # run all tests verbosely
```

5.5 Pytest Fixture Scopes

Scope	Runs	Use When
function	Before every test	Each test needs fresh state
module	Once per file	File-level shared setup
session	Once per test run	Tests depend on shared data

5.6 Deployment

```
# Production command
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

```
# Procfile (for Railway/Render)
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

```
# Switch to PostgreSQL for production (database.py)
from dotenv import load_dotenv
import os

load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL") # set on server
engine = create_engine(DATABASE_URL)
```

Platform	Free Tier	Difficulty
Railway	Yes	Easiest
Render	Yes	Easy
Fly.io	Yes	Medium
AWS/GCP	No	Complex

5.7 Edge Cases — Stage 5

Test naming: Files must start with `test_`. Functions must start with `test_`.

Separate test DB: Always use a different DB for tests. Never test against production DB.

dependency_overrides: FastAPI's way to swap dependencies (like `get_db`) for testing.

SQLite in production: File-based, doesn't handle multiple workers. Use PostgreSQL in production.

pip freeze: Always update `requirements.txt` before deploying.

--reload: Development only. Never use in production.

Quick Reference

Common Imports Cheatsheet

```
from fastapi import FastAPI, Depends, HTTPException, Request
from fastapi.security import OAuth2PasswordBearer
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import DeclarativeBase, sessionmaker, Session, Mapped, mapped_column
from sqlalchemy import create_engine
from pydantic import BaseModel
from typing import Optional
from jose import JWTError, jwt
from passlib.context import CryptContext
from datetime import datetime, timedelta, timezone
from dotenv import load_dotenv
import os, pytest
```

Key Patterns

```
# Protect a route
@app.get('/protected')
def protected(current_user = Depends(get_current_user)):
    return {"user": current_user}

# List response model
@app.get('/users', response_model=list[UserOut])
def get_users(db: Session = Depends(get_db)):
    return db.query(User).all()

# Use data-driven code instead of if-chains
greetings = {"en": "Hello", "es": "Hola", "fr": "Bonjour"}
if lang not in greetings:
    raise HTTPException(status_code=400, detail="Unsupported language")
return {"message": f"{greetings[lang]}, {name}!"}

# Override dependency in tests
app.dependency_overrides[get_db] = override_get_db
```

Alembic Quick Commands

```
alembic init alembic
alembic revision --autogenerate -m "description"
alembic upgrade head
alembic downgrade -1
alembic history
```